

Large-Scale Data Sets Clustering Based on MapReduce and Hadoop

Ping ZHOU[†], Jingsheng LEI, Wenjun YE

School of Computer and Information Engineering, Shanghai University of Electric Power, Shanghai 200090, China

Abstract

MapReduce is a simplified programming model of distributed parallel computing. It is an important technology of Google, and is commonly used for data-intensive distributed parallel computing. Cluster analysis is the most important data mining methods. Efficient parallel algorithms and frameworks are the key to meeting the scalability and performance requirements entailed in such scientific data analyses. In order to improve the deficiency of the long time in large-scale data sets clustering on the single computer, we design and realize a parallel K- Means algorithm based on MapReduce framework. This algorithm runs on Hadoop cluster. The results show that the MapReduce framework K-Means clustering algorithm can obtain a higher performance when handling large-scale document automatic classification, and they prove the effectiveness and accuracy of the algorithm.

Keywords: MapReduce; Hadoop; Parallel K-Means Clustering; Large-Scale data Sets

1. Introduction

Cluster analysis is one of the most important data mining methods. Document clustering is the act of collecting similar documents into classes, where similarity is some function on a document. Document clustering does not need separate training process, nor manual tagging group in advance. The documents in the same clusters are more similar, while the documents in different clusters are more dissimilar.

MapReduce is a parallel programming technique derived from the functional programming concepts and is proposed by Google for large-scale data processing in a distributed computing environment. Hadoop project[1] provides a distributed file system(HDFS). Hadoop is a Java-based software framework that enables data-intensive application in a distributed environment. Hadoop enables applications to work with thousands of nodes and terabyte of data, without concerning the user with too much detail on the allocation and distribution of data and calculation. Hadoop and MapReduce[2] realize the mass data storage, analysis and exchange management technology.

In this paper, we introduce MapReduce parallel programming model; propose a MapReduce and the Hadoop distributed clustering algorithm, the design and the implementation of the large-scale data processing model based on MapReduce and Hadoop; and give some experimental results of the distributed clustering based on MapReduce and Hadoop , and discuss the results.

[†] Corresponding author.

Email addresses: zhouping5460@126.com (Ping ZHOU).

2. Background

2.1. Hadoop Overview

When data sets go beyond a single storage capacity, it is necessary to distribute them to multiple independent computers. Trans-computer network storage file management system is called distributed file system. A typical Hadoop distributed file system contains thousands of servers, each server stores partial data of file system. HDFS cluster configuration is simple. It just needs more servers and some simple configuration to improve the Hadoop cluster computing power, storage capacity and IO bandwidth. In addition HDFS achieves reliable data replication, fast fault detection and automatic recovery, etc..

2.2. MapReduce Overview

In a distributed data storage, when parallel processing the data, we need to consider much, such as synchronization, concurrency, load balancing and other details of the underlying system. It makes the simple calculation become very complex. MapReduce programming model [3] was proposed in 2004 by the Google, which is used in processing and generating large data sets implementation. This framework solves many problems, such as data distribution, job scheduling, fault tolerance, machine to machine communication, etc.

MapReduce is applied in Google's Web search. Programmers need to write many programs for the specific purpose to deal with the massive data distributed and stored in the server cluster, such as crawled documents, web request logs, etc., in order to get the results of different data, such as inverted indices, web document, different views, worms collected the number of pages for each host a summary of a given date within the collection of the most common queries and so on.

3. MapReduce Programming Model

MapReduce programming model, by map and reduce function realize the Mapper and Reducer interfaces. They form the core of task.

3.1. Mapper

Map function requires the user to handle the input of a pair of key value and produces a group of intermediate key and value pairs. <key,value> consists of two parts, value stands for the data related to the task, key stands for the "group number " of the value . MapReduce combine the intermediate values with same key and then send them to reduce function.

Map algorithm process is described as follows:

Step1. Hadoop and MapReduce framework produce a map task for each InputSplit, and each InputSplit is generated by the InputFormat of job. Each <Key,Value> corresponds to a map task.

Step2. Execute Map task, process the input <key,value> to form a new <key,value>. This process is called "divide into groups". That is, make the correlated values correspond to the same key words. Output key value pairs that do not required the same type of the input key value pairs. A given input value pair can be mapped into 0 or more output pairs.

Step3. Mapper's output is sorted to be allocated to each Reducer. The total number of blocks and the

number of job reduce tasks is the same. Users can implement Partitioner interface to control which key is assigned to which Reducer.

3.2. Reducer

Reduce function is also provided by the user, which handles the intermediate key pairs and the value set relevant to the intermediate key value. Reduce function mergers these values, to get a small set of values. the process is called "merge ". But this is not simple accumulation. There are complex operations in the process. Reducer makes a group of intermediate values set that associated with the same key smaller.

In MapReduce framework, the programmer does not need to care about the details of data communication, so $\langle \text{key}, \text{value} \rangle$ is the communication interface for the programmer in MapReduce model. $\langle \text{key}, \text{value} \rangle$ can be seen as a "letter", key is the letter's posting address, value is the letter's content. With the same address letters will be delivered to the same place. Programmers only need to set up correctly $\langle \text{key}, \text{value} \rangle$, MapReduce framework can automatically and accurately cluster the values with the same key together.

Reducer algorithm process is described as follows:

Step1. Shuffle. Input of Reducer is the output of sorted Mapper. In this stage, MapReduce will assign related block for each Reducer.

Step2. Sort. In this stage, the input of reducer is grouped according to the key (because the output of different mapper may have the same key). The two stages of Shuffle and Sort are synchronized;

Step3. Secondary Sort. If the key grouping rule in the intermediate process is different from its rule before reduce. we can define a Comparator. The comparator is used to group intermediate keys for the second time.

Map tasks and Reduce task is a whole, can not be separated. They should be used together in the program. We call a MapReduce the process as an MR process. In an MR process, Map tasks run in parallel, Reduce tasks run in parallel, Map and Reduce tasks run serially. An MR process and the next MR process run in serial, synchronization between these operations is guaranteed by the MR system, without programmer's involvement.

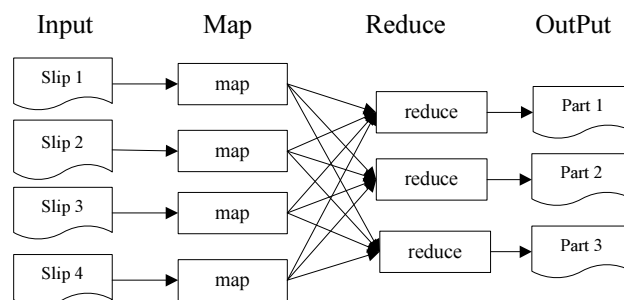


Fig. 1 MapReduce Operation Process

4. MapReduce Parallel K-Means Based Clustering Algorithm

4.1. Document Vector Representation

Vector space model is the most widely used method in information retrieval. This model uses feature entries and their weights to express the document information. Vector $d = (w_1, w_2, w_3, \dots, w_m)$ stands for the feature entry and its weight in document d , m is the number of all feature entries. $w_i (i = 1, \dots, m)$ is the weight of the entry t_i in document d . The document vector set is the pattern or data object of the document clustering.

Web document vector similarity function is a cosine function, whose returned value is $[0,1]$. The greater the returned value is, the greater the similarity is.

$$\sin(d_i, d_j) = \frac{w_{i1} * w_{j1} + \dots + w_{im} * w_{jm}}{|d_i| * |d_j|} \quad (1)$$

$(w_{i1}, \dots, w_{im})$ is the feature vector of the document d_i .

$$|d_i| = \sqrt{\sum_{k=1}^m w_{ik}^2}, \quad |d_j| = \sqrt{\sum_{k=1}^m w_{jk}^2}. \quad (2)$$

4.2. Document Preprocessing Based on MapReduce

We introduce the calculation of word frequency by MapReduce. When calculating TF, map function reads text. Each line of the text represents a document and its type. Key is the document's type, value is the document's content. In the Intermediate output data, keys are the document's type and the word in the document. Values are the frequency of the word in the document. Reduce function accumulates all the values with the same key to get the frequency of the word t in all documents. The calculation of DF, cluster number and the document corpus and the total number of words is similar to that of TF.

First, we need to preprocess the document. That is, extract every word of the document, and calculate its frequency. In the Map stage, when we meet a word, $\langle \text{words}, \text{"the document id, 1"} \rangle$. Is output. Hadoop platform ensures that all the values with the same key are assigned to the same reducer.

Doc_i figure above that documents, w_{ij} that word, n_{ij} is word frequency.

In the Map phase, each input of Map is a document type id_n and its content d . First, map function creates a set H , then calculate t the frequency in H . Finally, output all elements of H . The keys of output are pairs $\langle \text{word } t, \text{the document id } n \rangle$, values is the t frequency list.

In Reduce stage, every reduce receives a key value pair. *INITIALIZE* function creates a list *PostingList*, then processes each input. If meet a new word, output the existing list, otherwise, then add the document id and document frequency into the list *PostingList*. Finally, output all the words and their corresponding *PostingList*.

4.3. K-Means Clustering Methods

K-Means algorithm[4,5,6] is widely used clustering algorithm. Firstly, the algorithm randomly select k

initial objects. Each one represents a cluster center. The rest of the objects will be assigned to the nearest cluster, according to their distances to different centers. Then calculate every center again. This operation is repeated until the criterion function converges.

Algorithm description as following:

Input: The number of clusters k and n documents

Output: k clusters

Step1. Randomly select k documents from n documents as the initial cluster centers.

Step2. Calculate the distances of the rest documents to the every center of the clusters, and assign each of the rest documents to the nearest cluster.

Step3. Calculate and adjust each cluster center.

Step4. Iterate **Step2** ~ **Step3** until the criterion function converge. The program ends.

4.4. Parallel K-Means Clustering Algorithm Based on MapReduce and Hadoop

As shown in Figure 3, the MapReduce adaption of the algorithms [7] works as follows:

Step1. The first stage is the document preprocessing. we divide a data set D into m subsets. There are has two MapReduce jobs in this stage. The first job calculates the parameters that are required in the next step. The following MapReduce job calculates the DFR and $TFIDF$ of each term, and then extracts terms and generates VSM . We create a file which contains : iteration number、cluster id、cluster coordinates、number of documents assigned to the cluster.

Step2. The second stage is the map function, the distance between each document and each cluster center is calculated, then reads the input data and calculates the distance to each center. For each document, it produces an output pair with: $\langle key(cluster\ id), value(coordinates\ of\ documents) \rangle$. It produces a lot of data in this stage. we can use a combine function to reduce the size before sending it to Reduce. Combine function is described as following.

The Combine function calculates the average of the coordinates for each cluster id, along with the number of documents. All data of the same current cluster are sent to a single reducer.

Step3. The third stage is reduce function. In the reduce function, compute new cluster center coordinates. Its output is written to the cluster file, and contain: iteration number、cluster id、the cluster center coordinates、the size of the cluster.

Step4. At last the new cluster coordinates are compared to the original ones. If the criterion function converges, the program end, and we have found the clusters. If not, use the newly generated cluster centers and iterate **Step 2** to **4**.

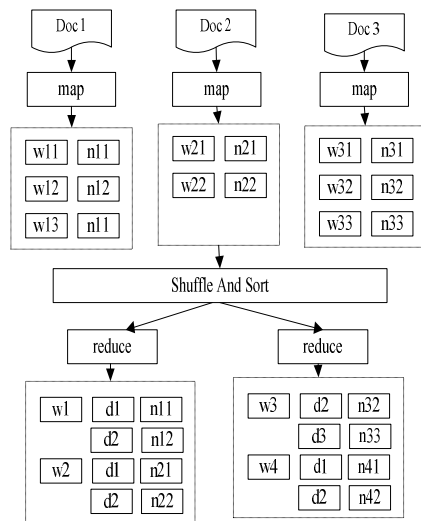


Fig. 2 Document Preprocessing Parallel Process

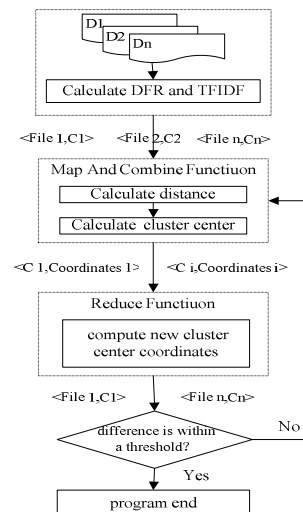


Fig. 3 Parallel K-Means Algorithm Process

5. Experimental Results

The data used in this experiment comes from text classification corpus of Sogou lab. The corpus is the news on *Sohu* site in recent years. Corpus data is divided into 10 categories (automobile, finance, IT, health, sports, tourism, education, recruitment, culture, and military). We choose a number of documents to cluster and verify the effectiveness and feasibility of the algorithm.

Experiment 1. Executing time by K-Means algorithm on stand-alone computer and based on MapReduce and Hadoop for the same number of documents.

The following table shows the executing time of stand-alone K-Means algorithm and the parallel one for 50,000 documents, about 50M data

Table 1 The Executing Result of 2 Different K-Means Algorithm

	Stand along	Hadoop and MapReduce
Clustering	K-Means clustering	Parallel K-Means clustering
The number of Documents	50000 (50M)	50000(50M)
The number of nodes	1	10
Execution time	30 minutes	10 minutes

Fig.4 The following figure shows for different numbers of documents, the executing time of stand-alone K-Means algorithm and the parallel one.

Experiment 2. Executing time of parallel K-Means algorithm based on Hadoop and MapReduce under different numbers of nodes.

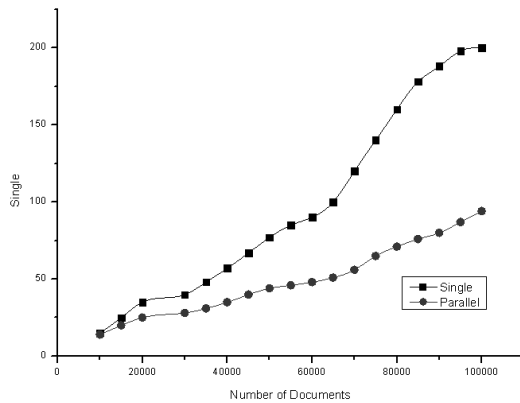


Fig. 4 The Executing Time of Stand-alone K-Means Algorithm and the Parallel One.

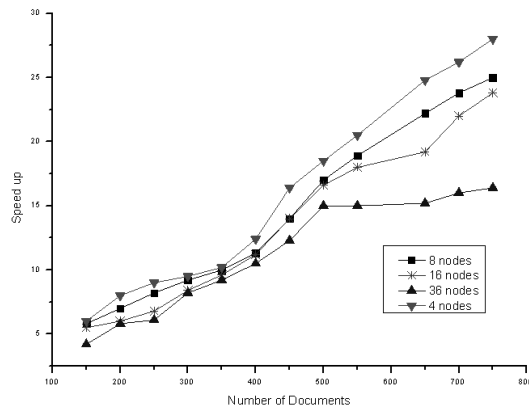


Fig. 5 Results of Parallel K-Means Algorithm under Different Numbers of Nodes.

Experiment 3. Experiment to verify the accuracy of document clustering.

The third experiment shows that, the average accuracy of the parallel K-Means algorithm is 89%, it verifies the effectiveness of the parallel algorithm. The result show in Table 2.

Table 2 Results of Clustering

ID	Name	Number of Cluster	Number of Cluster	Number of Cluster	Precision
C1	Technology	11	10	1	90.9%
C2	Finance	54	50	4	92.6%
C3	Education	25	23	2	92 %
C4	Estate	53	49	4	92.5%
C5	Sports	43	39	4	90.1%

6. Conclusions and Future Work

This work represents only a small first step in using the MapReduce programming technique in the process of large-scale data Sets.

We take the advantage of the parallellism of MapReduce to design a parallel K-Means clustering algorithm based on MapReduce. This algorithm can automatically cluster the massive data, making full use of the Hadoop cluster performance. It can finish the text clustering in a relatively short period of time. Experiments show that it achieves high accuracy.

The main deficiency: we have to set the number k (the number of clusters to be generated) in advance, and this algorithm is sensitive to initial values. Different initial values may lead to different results. And it is sensitive to the "noise" and the outlier data.

We need to optimize from three aspects to reduce the running time based on Hadoop platform:

The following three aspects can be optimized to reduce the level of the Hadoop cluster platform in the run-time.

- (1) Reduce program runs among multi-nodes instead of a single node.
- (2) Platform optimization, we have not optimized the platform yet. The platform spends extra

management time in each iteration. We can reduce platform management time.

(3) Optimization of the algorithm itself. We need to custom outputs of mapper and reducer functions. There is more efficient data format that the intermediate results can be stored. These options will enable faster read and write.

Acknowledgement

This work was supported by National Nature Science Foundation of China (No. 61073189).

References

- [1] Hadoop. <http://hadoop.apache.org/>.
- [2] S. Ghemawat, H. Gobioff, S.T. Leung, The Google file system, *19th Symposium on Operating Systems Principles, New York*, pages 29-43 2003.
- [3] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation*, pages 137-149, 2004.
- [4] H. Maulik, and S. Bandyopadhyay. Genetic Algorithm-Based Clustering Technique. *Pattern Recognition*, Vol.33, pages 1455-1465, 2000.
- [5] S.Z. Selim, and M.A. Ismail. K-means-Type Algorithms: a Generalized Convergence Theorem and Characterization of Local Optimality. *IEEE Trans on Pattern Analysis and Machine Intelligence*, Vol.6, No.1, pages 81-87, 1984.
- [6] D. Arthur and S. Vassilvitskii. K-Means++: The Advantage of Careful Seeding. *Society for Industrial and Applied Mathematics*, Philadelphia, PA, USA, 2007.
- [7] Berry, M. <http://blog.data-miners.com/2008/02/MapReduce-and-k-means-clustering.html>, 2008.